INTERNATIONAL UNIVERSITY

SCHOOL OF COMPUTER SCIENCE & ENGINEERING

Web Application Development

PROJECT REPORT

Project Information and Contributors

Project Name:

Hotel management system

Contribution and Information of Each Member:

Team member name	Student ID	Individual overall work contribution (%)	Task
Phan Huy Thien Tan	ITCSIU22132	33	Developed the logic controller and security system
Tran Duc Huy	ITITWE21116	33	Design the model and repository . Do a report documents and slides presentation
Phan Tran Kim Bao (Leader)	ITITWE24004	33	Design the UI of the project and testing
Total 100%			

Table of Contents

1. Executive Summary

2. Requirements Analysis

- 2.1 Problem Statement
- 2.2 Functional Requirements
- 2.3 Non-Functional Requirements

3. System Architecture

- 3.1 Architecture Overview
- 3.2 Technology Stack Justification
- 3.3 Design Patterns
- 3.4 API Design

4. Database Design

- 4.1 Entity Relationships
- 4.2 Table Schemas
- 4.3 Relationships Explanation
- 4.4 Normalization Analysis
- 4.5 Indexing Strategy

5. Implementation Details

- 5.1 Key Algorithms & Business Logic
- 5.2 Security Implementation
- 5.3 Error Handling
- 5.4 Advanced Features

6. Testing & Quality Assurance

- 6.1 Testing Strategy
- 6.2 Known Limitations

7. Deployment

- 7.1 Deployment Process
- 7.2 CI/CD
- 7.3 Monitoring

8. User Guide

- 8.1 Getting Started
- 8.2 Feature Walkthroughs

9. Conclusion & Reflection

- 9.1 Summary
- 9.2 Lessons Learned
- 9.3 Future Enhancements

1. Executive Summary

The Hotel Management System is a full-stack web application built with Spring Boot that centralises all hotel operations — from guest check-in to housekeeping and billing — into a single, role-aware platform.

1.1 Project Overview

Modern hotels rely on multiple disconnected tools to manage guests, rooms, food orders, housekeeping and billing. This project delivers a unified web system that allows receptionists, housekeeping staff, food service workers and administrators to collaborate in real time through a single application backed by a relational database.

Key Features

Guest Management: Register guests with ID verification (Passport, National ID, Driving Licence). Auto-lookup existing guests on re-visit.

Room Booking: Check date-conflict-safe reservation logic with RESERVED → CHECKED_IN → CHECKED_OUT → CANCELLED lifecycle.

Check-in / Check-out: Real-time status updates; room automatically transitions to OCCUPIED or CLEANING on state change.

Billing & Invoicing: Automatic invoice generation on check-out combining room charges (nights × rate) and food charges.

Food Order Tracking: In-room dining orders with PENDING → PREPARING → DELIVERED workflow.

Housekeeping Management: Task assignment (CLEANING / INSPECTION / MAINTENANCE) with staff tracking.

Inventory Control: Low-stock alerts and quantity tracking for linens, toiletries, food & cleaning supplies.

Role-Based Access Control: Four roles: ADMIN, RECEPTIONIST, HOUSEKEEPING, FOOD_SERVICE — each with scoped permissions.

Dashboard Analytics: Live occupancy rate, revenue summary, pending tasks, and low-stock alerts.

Technology Stack

Layer	Technology
Backend Framework	Spring Boot
Language	Java
ORM	Spring Data JPA / Hibernate
Security	Spring Security + BCrypt
View Layer	Thymeleaf
Database	MySQL
Build Tool	Maven
Utilities	Lombok

2. Requirements Analysis

2.1 Problem Statement

Small to mid-size hotels typically manage operations through spreadsheets, paper ledgers, or a patchwork of disconnected tools. This creates friction at every touchpoint: receptionists manually check room availability, housekeeping communicates via phone calls, billing requires manual calculation, and management has no real-time view of occupancy or revenue. Errors are common: double-bookings, lost food orders, uncleaned rooms assigned to new guests, and billing discrepancies that damage guest trust.

The Hotel Management System addresses these pain points by providing a single, role-aware web application that keeps all data consistent and accessible in real time.

Target Users

Hotel Administrators — manage staff accounts, view reports, configure room categories.

Receptionists — handle guest check-in / check-out, reservations, and billing.

Housekeeping Staff — view assigned cleaning / maintenance tasks and update status. Food Service Staff — receive and fulfill in-room dining orders.

Hotel Guests — indirect users; their data is captured and managed by staff.

2.2 Functional Requirements

Authentication & Authorisation

Staff can log in with email + password via a form login page. BCrypt-hashed passwords; plain-text passwords never stored.

Role-based access: ADMIN, RECEPTIONIST, HOUSEKEEPING, FOOD_SERVICE.

API endpoints protected by role annotations (@PreAuthorize / hasRole).

Guest Management

Create, read, update guest records with full_name, email, phone, id_type, id_number. getOrCreate semantics: return existing guest if email matches to avoid duplicates.

Input validation via Jakarta Bean Validation (@NotBlank, @Email).

Room & Category Management

Define room categories with name, description, price_per_night, max_occupancy. Rooms belong to a category; each room has a floor and a status.

Room statuses: AVAILABLE, OCCUPIED, MAINTENANCE, CLEANING.

Booking Lifecycle

Create booking: validate room AVAILABLE + no date overlap, then set RESERVED. Check-in: transition RESERVED → CHECKED_IN; room → OCCUPIED.

Check-out: transition CHECKED_IN → CHECKED_OUT; room → CLEANING; auto-generate bill. Cancel: only RESERVED bookings may be cancelled.

Billing

Bill auto-created on check-out: room_charges = nights × category.pricePerNight. Food charges aggregated from all DELIVERED food orders for the booking.

Bill statuses: UNPAID (default), PAID (after payment confirmation).

Food Orders

Link food orders to a booking; track item_name, quantity, unit_price.

Order lifecycle: PENDING → PREPARING → DELIVERED. Total price computed at runtime (@Transient) to avoid stale data.

Housekeeping

Create tasks of type CLEANING, INSPECTION, or MAINTENANCE linked to a room and staff. Task lifecycle: PENDING → IN_PROGRESS → DONE. Completion timestamp recorded automatically when status set to DONE.

Inventory

Track items with category, quantity, unit, and low_stock_threshold. isLowStock() helper returns true when quantity ≤ threshold. Dashboard surfaces low-stock alerts for ADMIN role.

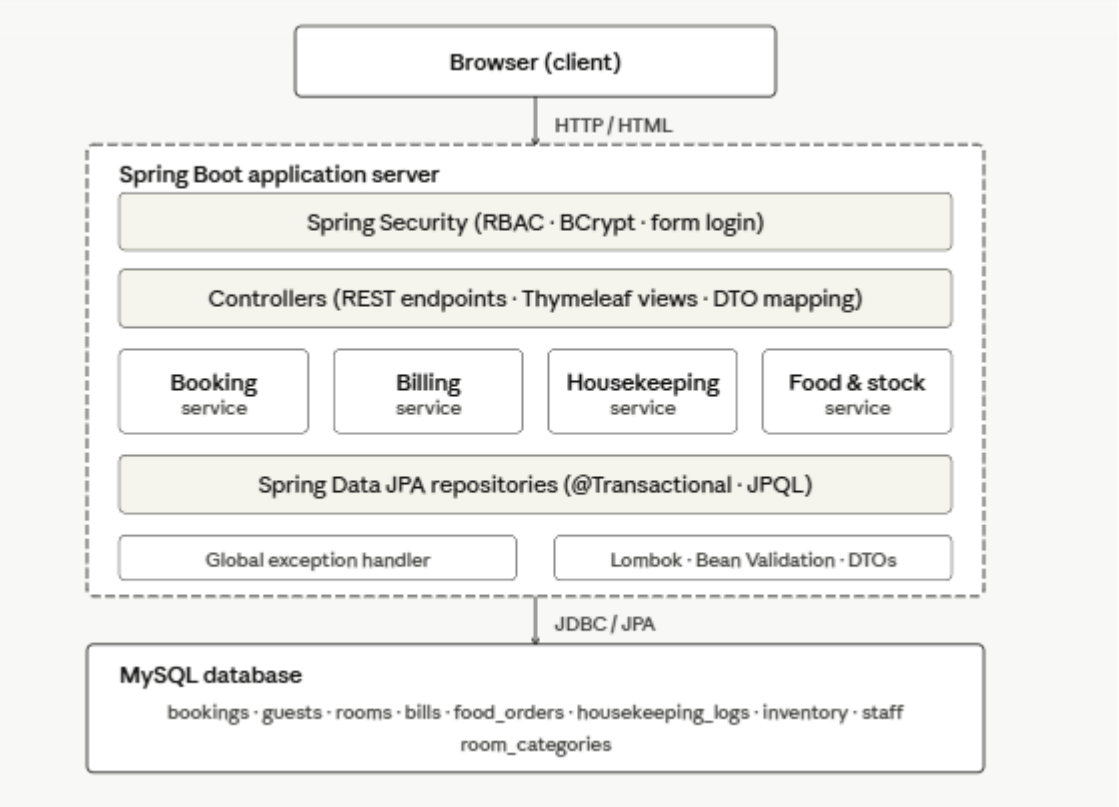
2.3 Non-Functional Requirements

Category	Requirement
Performance	API response time for read endpoints
Performance	Page load time (Thymeleaf rendered)
Security	Password storage
Security	SQL injection prevention
Security	CSRF (dev mode disabled; production must enable)
Usability	Responsive layout
Usability	Form validation feedback
Scalability	Concurrent users
Reliability	Uptime target
Reliability	Transactional integrity

3. System Architecture

3.1 Architecture Overview

The system follows a classic three-tier architecture: a browser-based presentation layer rendered by Thymeleaf, a Spring Boot application server handling all business logic, and a MySQL relational database for persistent storage. All layers run on the same JVM process for simplicity, but the clean package separation makes extraction straightforward.



3.2 Technology Stack Justification

Spring Boot over Raw Servlets / JAX-RS

Spring Boot provides auto-configuration, an embedded Tomcat server, and opinionated defaults that eliminate boilerplate. Raw Servlets require manual configuration of every filter, datasource, and transaction manager. Spring Boot reduces bootstrapping from hundreds of lines to a single @SpringBootApplication annotation.

Spring Data JPA over JDBC Template

JPA with Hibernate lets us define database relationships directly in Java classes (@ManyToOne, @OneToOne) and generates SQL automatically. JDBC Template requires hand-written SQL and manual ResultSet mapping. JPA also provides built-in transaction management and lazy loading.

MySQL over MongoDB

Hotel data is highly relational: a booking depends on a guest and a room; a bill depends on a booking. A relational database with foreign key constraints prevents orphaned records and data inconsistency. MongoDB's flexible schema would suit unstructured documents but adds complexity when joining related entities.

Thymeleaf over React / Vue SPA

Thymeleaf is a natural fit for a server-side Spring MVC application — templates are rendered on the server and returned as complete HTML. A separate SPA would require building and maintaining a dedicated REST API consumed by JavaScript, doubling development effort for a staff-facing internal tool.

Spring Security over Custom Auth

Spring Security provides a production-grade filter chain, BCrypt password hashing, CSRF protection, method-level security (@PreAuthorize), and seamless integration with Spring MVC. Building equivalent authentication from scratch would be error-prone and time-consuming.

Lombok over Boilerplate

Lombok's @Data annotation auto-generates getters, setters, equals(), hashCode() and toString() at compile time via annotation processing. This keeps model classes concise and focused on their fields while ensuring consistency.

3.3 Design Patterns

Model-View-Controller (MVC)

Controllers receive HTTP requests and delegate to services. Services contain business logic. Models represent the data. Thymeleaf templates are the view. Spring MVC enforces this separation via @Controller, @Service, and @Entity annotations.

Repository Pattern

All database access is abstracted behind Repository interfaces that extend JpaRepository. Controllers and services never write SQL directly, which decouples business logic from persistence and simplifies testing.

Service / Facade Pattern

Service interfaces (BookingService, BillService, etc.) expose high-level operations to controllers. The concrete implementations (BookingServiceImpl) coordinate multiple repositories and enforce business rules, keeping controllers thin.

DTO Pattern

BookingRequestDTO / BookingResponseDTO separate the external API contract from the internal entity model. This prevents over-posting attacks, allows different views of the same data, and isolates API changes from schema changes.

Template Method (Lifecycle Transitions)

checkIn() and checkOut() follow a consistent pattern: validate current status → update status → update related entity (room) → persist → trigger side effect (bill). This consistent structure makes the lifecycle easy to reason about and extend.

3.4 API Design

The system exposes a RESTful JSON API under /api/* for each domain entity. All endpoints follow the
/api/{resource}/{id} convention and use standard HTTP verbs. Responses use appropriate HTTP status codes.

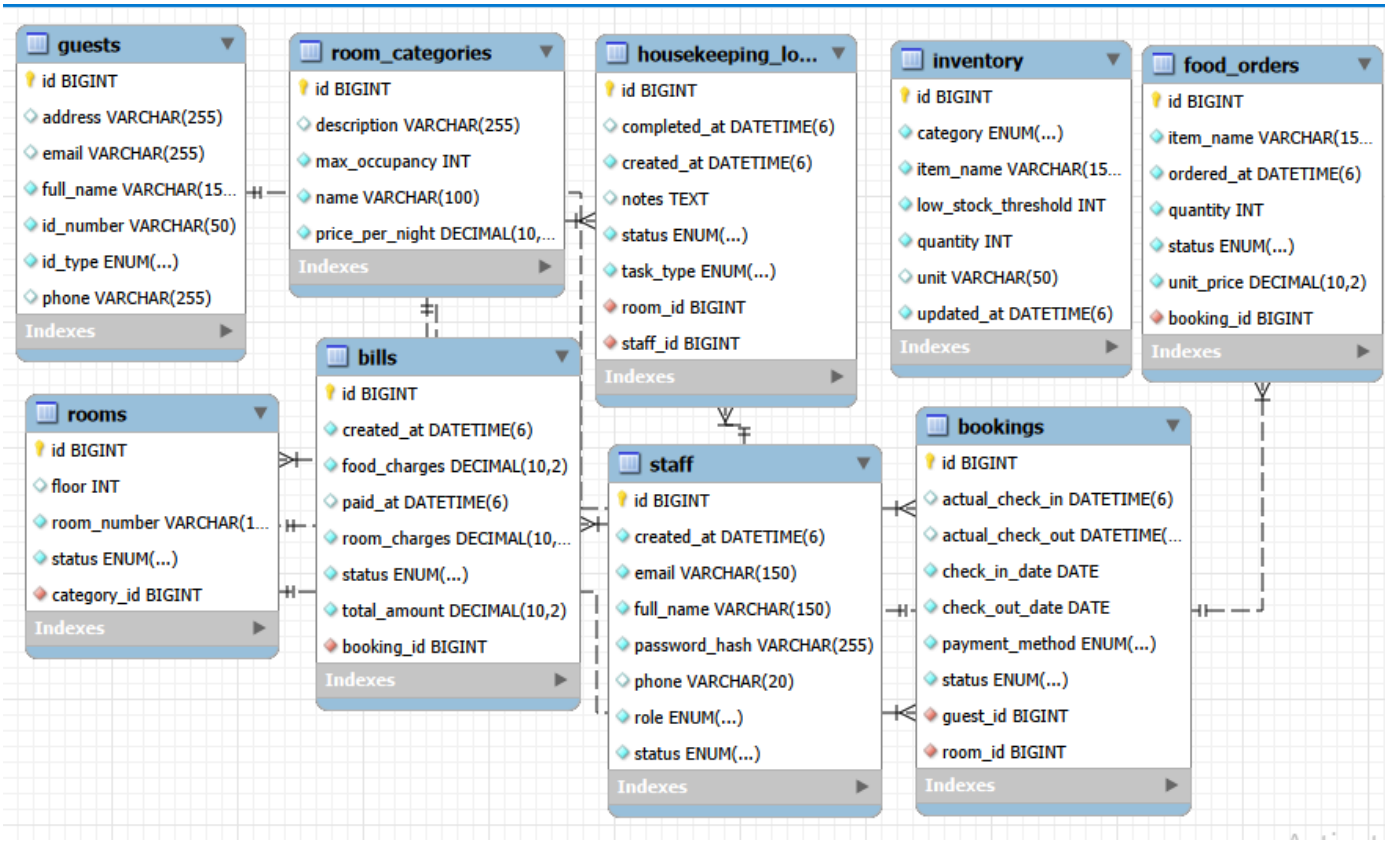
Method	URL	Description	Auth Role
GET	/api/bookings	List all bookings	Any
POST	/api/bookings	Create new booking	RECEPTIONIST, ADM
GET	/api/bookings/{id}	Get booking by ID	Any
PUT	/api/bookings/{id}/checkin	Perform check-in	RECEPTIONIST, ADM
PUT	/api/bookings/{id}/checkout	Perform check-out + generate bill	RECEPTIONIST, ADM
PUT	/api/bookings/{id}/cancel	Cancel reservation	RECEPTIONIST, ADM

PUT	/api/bills/{id}/pay	Mark bill as paid	RECEPTIONIST, ADM
GET	/api/staff	List staff members	ADMIN only
POST	/api/staff	Create staff account	ADMIN only
GET	/api/housekeeping	List housekeeping tasks	HOUSEKEEPING, AD
PUT	/api/housekeeping/{id}/status	Update task status	HOUSEKEEPING, AD
GET	/api/food-orders	List food orders	FOOD_SERVICE, AD
PUT	/api/food-orders/{id}/status	Update order status	FOOD_SERVICE, AD

4. Database Design

4.1 Entity-Relationship Overview

The database consists of nine tables. The central entity is **bookings**, which links guests to rooms and serves as the anchor for bills and food orders. Staff records are independent but linked to housekeeping tasks.



Key Relationships

room_categories (1) → (N) rooms — each room belongs to exactly one category. guests (1) → (N) bookings — a guest can have multiple bookings over time.

rooms (1) → (N) bookings — a room can be booked many times (non-overlapping). bookings (1) → (1) bills — every checked-out booking has exactly one bill. bookings (1) → (N) food_orders — a booking can have many food orders.

rooms (1) → (N) housekeeping_logs — a room can have many cleaning tasks over time. staff (1) → (N) housekeeping_logs — a staff member can be assigned many tasks.

4.2 Table Schemas

Table: staff

Column	Type	Constraints

id	BIGINT	PRIMARY KEY, AUTO_INCREMENT
full_name	VARCHAR(150)	NOT NULL
email	VARCHAR(150)	UNIQUE NOT NULL
password_hash	VARCHAR(255)	NOT NULL — BCrypt hash, never plain text
role	ENUM	NOT NULL — ADMIN | RECEPTIONIST | HOUSEKEEPING | FOOD_SERVICE
phone	VARCHAR(20)	NULLABLE
status	ENUM	NOT NULL DEFAULT ACTIVE — ACTIVE | INACTIVE

Column	Type	Constraints
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP, NOT UPDATABLE

Table: guests

Column	Type	Constraints
id	BIGINT	PRIMARY KEY, AUTO_INCREMENT
full_name	VARCHAR(150)	NOT NULL
email	VARCHAR(255)	NULLABLE (indexed for lookup)
phone	VARCHAR(30)	NULLABLE
id_type	ENUM	NOT NULL — PASSPORT | NATIONAL_ID | DRIVING_LICENSE
id_number	VARCHAR(50)	NOT NULL
address	VARCHAR(500)	NULLABLE

Table: room_categories

Column	Type	Constraints
id	BIGINT	PRIMARY KEY, AUTO_INCREMENT
name	VARCHAR(100)	NOT NULL — e.g. Standard, Deluxe, Suite
description	TEXT	NULLABLE
price_per_night	DECIMAL(10,2)	NOT NULL
max_occupancy	INT	NOT NULL DEFAULT 2

Table: rooms

Column	Type	Constraints
id	BIGINT	PRIMARY KEY, AUTO_INCREMENT
room_number	VARCHAR(10)	UNIQUE NOT NULL
category_id	BIGINT	FK → room_categories.id, NOT NULL
floor	INT	NULLABLE
status	ENUM	NOT NULL DEFAULT AVAILABLE — AVAILABLE | OCCUPIED | MAINTENANCE | CLEANING

Table: bookings

Column	Type	Constraints
id	BIGINT	PRIMARY KEY, AUTO_INCREMENT
guest_id	BIGINT	FK → guests.id, NOT NULL

Column	Type	Constraints
room_id	BIGINT	FK → rooms.id, NOT NULL
check_in_date	DATE	NOT NULL
check_out_date	DATE	NOT NULL
actual_check_in	TIMESTAMP	NULLABLE — set on check-in action
actual_check_out	TIMESTAMP	NULLABLE — set on check-out action
payment_method	ENUM	NOT NULL — ONLINE | CASH
status	ENUM	NOT NULL DEFAULT RESERVED — RESERVED | CHECKED_IN | CHECKED_OUT | CANCELLED

Table: bills

Column	Type	Constraints
id	BIGINT	PRIMARY KEY, AUTO_INCREMENT
booking_id	BIGINT	FK → bookings.id, UNIQUE NOT NULL
room_charges	DECIMAL(10,2)	NOT NULL
food_charges	DECIMAL(10,2)	NOT NULL DEFAULT 0.00
total_amount	DECIMAL(10,2)	NOT NULL — computed: room + food
status	ENUM	NOT NULL DEFAULT UNPAID — UNPAID | PAID
paid_at	TIMESTAMP	NULLABLE
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP

4.3 Normalization Analysis

The schema is in Third Normal Form (3NF). All non-key attributes depend only on the primary key (2NF), and there are no transitive dependencies (3NF). Key design decisions that demonstrate normalization:

Room price is stored in room_categories, not rooms. Many rooms share one category price, so repeating it per room would create update anomalies.

total_amount in bills is derived (room_charges + food_charges). It is stored explicitly because it is the official billed amount at check-out time; storing it prevents retrospective changes if prices are updated later.

food_orders stores unit_price at order time (not a FK to a menu). This correctly captures the historical price, not the current price, for accurate billing.

getTotalPrice() and isLowStock() are computed as @Transient properties — they are never stored because they can always be derived and storing them would create redundancy.

The guest table is separate from the booking table (normalised). One guest row is reused across multiple bookings via getOrCreate logic.

4.4 Indexing Strategy

guests	email	INDEX	getOrCreate lookup by email.
rooms	room_number	UNIQUE INDEX	Uniqueness + fast lookup.
bookings	guest_id	INDEX (FK)	Filter bookings by guest.
bookings	room_id	INDEX (FK)	Date-conflict check query.
bookings	status	INDEX	Dashboard queries by status.
bills	booking_id	UNIQUE INDEX (FK)	One-to-one lookup.
food_orders	booking_id	INDEX (FK)	Aggregate food charges per booking.
housekeeping_log	sstaff_id	INDEX (FK)	Tasks assigned to a staff member.
inventory	item_name	UNIQUE INDEX	Duplicate prevention.

5. Implementation Details

5.1 Key Algorithms & Business Logic

A. Booking Conflict Detection

The most critical booking rule is preventing double-booking. A naive check would only look at exact date equality, missing overlaps where one booking starts before another ends. The system uses an interval-overlap query:

```
    // check if a room is already booked for given dates (for conflict check)
    @Query("""
        SELECT COUNT(b) > 0 FROM Booking b
        WHERE b.room.id = :roomId
        AND b.status != 'CHECKED_OUT'
        AND b.checkInDate < :checkOut
        AND b.checkOutDate > :checkIn
    """)
    boolean isRoomBooked(
        @Param("roomId")   Long roomId,
        @Param("checkIn")  LocalDate checkIn,
        @Param("checkOut") LocalDate checkOut
    );
}
```

The condition checkInDate < :checkOut AND checkOutDate > :checkIn is the standard interval-overlap test. Two date ranges [A,B) and [C,D) overlap if and only if A < D and C < B. CANCELLED bookings are excluded from the check, but CHECKED_OUT bookings are also excluded because a room can be rebooked after a guest departs.

B. Automatic Bill Generation on Check-out

```
    @Override
    public Bill createBillForBooking(Booking booking) {
        // Calculate nights spent
        long days = ChronoUnit.DAYS.between(booking.getCheckInDate(), booking.getCheckOutDate());
        if (days <= 0) {
            days = 1; // charge at least 1 night
        }

        BigDecimal pricePerNight = booking.getRoom().getCategory().getPricePerNight();
        BigDecimal roomCharges = pricePerNight.multiply(BigDecimal.valueOf(days));

        // Get total food charges
        BigDecimal foodCharges = foodOrderRepository.getTotalFoodChargesByBooking(booking.getId());
        if (foodCharges == null) {
            foodCharges = BigDecimal.ZERO;
        }

        // Check if bill already exists
        Bill bill = billRepository.findByBookingId(booking.getId()).orElse(null);
        if (bill == null) {
            bill = new Bill();
            bill.setBooking(booking);
        }

        bill.setRoomCharges(roomCharges);
        bill.setFoodCharges(foodCharges);
```

```
        bill.setRoomCharges(roomCharges);
        bill.setFoodCharges(foodCharges);
        bill.calculateTotal();
        bill.setStatus(Bill.BillStatus.UNPAID);

        return billRepository.save(bill);
    }

    @Override
    public Bill payBill(Long billId) {
        Bill bill = getBillById(billId);
        bill.setStatus(Bill.BillStatus.PAID);
        bill.setPaidAt(LocalDateTime.now());
        return billRepository.save(bill);
    }

    @Override
    public Bill getBillById(Long id) {
        return billRepository.findById(id)
                .orElseThrow(() -> new ResourceNotFoundException("Bill not found with ID: " + id));
    }
}
```

BigDecimal is used throughout billing (not double or float) to avoid floating-point rounding

errors. The food charges are aggregated in a single SQL query rather than loading all food order objects into memory.

C. Room Status State Machine

Room status transitions are enforced by the service layer, not the database. The valid transitions are: AVAILABLE → OCCUPIED (on check-in)
OCCUPIED → CLEANING (on check-out)
CLEANING → AVAILABLE (when housekeeping marks task DONE) AVAILABLE / CLEANING → MAINTENANCE (manual admin action) MAINTENANCE → AVAILABLE (when maintenance task completed)

D. Guest getOrCreate Logic

When creating a booking, the system checks whether a guest with the same email already exists. If found, the existing record is reused; otherwise a new guest is created. This prevents duplicate guest records for returning customers while keeping the booking creation flow seamless.

E. Transient Computed Properties

FoodOrder.java — total price computed, never persisted @Transient public BigDecimal getTotalPrice() { return unitPrice.multiply(BigDecimal.valueOf(quantity)); } // Inventory.java — low stock flag computed, never persisted @Transient public boolean isLowStock() { return this.quantity <= this.lowStockThreshold; }

Marking derived properties as @Transient prevents JPA from trying to map them to database columns. Values are computed at runtime from stored fields, ensuring they are always consistent and never stale.

5.2 Security Implementation

Password Hashing: BCryptPasswordEncoder with default strength 10 (approx. 100ms per hash on modern hardware). Passwords are hashed before storage; the plain-text password is never retained. **SQL Injection Prevention:** All queries use JPA / JPQL with @Param named parameters. No string concatenation is used to build queries. JPA's parameterised queries prevent injection by treating user input as data, not SQL.

XSS Prevention: Thymeleaf auto-escapes all output by default using th:text. HTML injection is prevented at the template level without requiring explicit escaping in controller code.

Role-Based Authorisation: Spring Security's HttpSecurity rules restrict endpoint access by role. @EnableMethodSecurity enables @PreAuthorize on individual service methods for fine-grained control.

Password Field Protection: @JsonIgnore on Staff.passwordHash ensures the hash is never serialised to JSON responses, preventing accidental exposure via API endpoints.

CSRF Note: CSRF is disabled for local development convenience. Production deployment must re-enable it. Spring Security's CSRF filter generates a per-session token and validates it on mutating requests.

5.3 Error Handling Strategy

The centralised @RestControllerAdvice class catches all exceptions thrown from any controller or service. This means service methods can throw meaningful exceptions (ResourceNotFoundException, IllegalStateException) without any try-catch in the controller layer. The ErrorResponseDTO provides a consistent JSON structure with status code, error type, human-readable message, and the request URI.

6. Testing & Quality Assurance

6.1 Testing Strategy

Testing was conducted manually via the application UI and the REST API. The following scenarios were verified:

ID	Scenario	Expected Result
TC-01	Register new guest with valid data	Guest record created, returned in API response
TC-02	Register guest with duplicate email	Existing guest returned (getOrCreate), no duplicate
TC-03	Book available room for future dates	Booking created with RESERVED status
TC-04	Book room already reserved for same dates	400 Bad Request: already booked
TC-05	Book room with status OCCUPIED	400 Bad Request: room not available
TC-06	Check-in RESERVED booking	Status → CHECKED_IN; room → OCCUPIED
TC-07	Check-in already CHECKED_IN booking	400 Bad Request: invalid status transition
TC-08	Check-out CHECKED_IN booking	Status → CHECKED_OUT; room → CLEANING; bill generated
TC-09	Check-out without prior check-in	400 Bad Request: cannot check-out from RESERVED
TC-10	Cancel RESERVED booking	Status → CANCELLED
TC-11	Cancel CHECKED_IN booking	400 Bad Request: only RESERVED can be cancelled
TC-12	View bill after check-out with food orders	Bill shows correct room + food total
TC-13	View bill before check-out	404 Not Found: bill does not exist yet
TC-14	HOUSEKEEPING role access to /api/staff	403 Forbidden (ADMIN only)
TC-15	Inventory item reaches low-stock threshold	isLowStock() returns true in response
TC-16	Create food order linked to active booking	Order created with PENDING status
TC-17	Progress food order through lifecycle	PENDING → PREPARING → DELIVERED
TC-18	Create housekeeping task and complete it	Task DONE; completedAt timestamp recorded

6.2 Known Limitations

No automated tests: JUnit/Mockito tests are not yet implemented. All testing is manual. Future work should add unit tests for service methods (BookingServiceImpl, BillServiceImpl) and integration tests for API endpoints.

CSRF disabled: CSRF protection is disabled for development convenience. Production deployment must re-enable it in SecurityConfig.

No email notifications: Booking confirmation emails are not sent. Guests receive no digital record of their reservation.

Single-server architecture: The application runs as a single Spring Boot instance with no load balancing. Horizontal scaling would require session management changes (sticky sessions or JWT-based stateless auth).

No audit log: No history of who changed what. A production system should record who performed each check-in, check-out, and status change for accountability.

Room conflict excludes CANCELLED only: CHECKED_OUT bookings are excluded from conflict checks, but the current check does not handle edge cases where a room might be double-booked if a booking is manually reinstated.

7. Deployment

7.1 Deployment Process

1. **Prerequisites:** Java 17 JDK, Maven 3.8+, MySQL 8.x server running.
2. **Database setup:** Create a MySQL database: CREATE DATABASE hotel_db; Grant privileges to the application user.
3. **Configure application.properties:** Set spring.datasource.url, spring.datasource.username, spring.datasource.password. Set spring.jpa.hibernate.ddl-auto=update for first run (creates tables automatically).
4. **Seed data:** DatabaseSeeder.java seeds default admin account, room categories, and sample rooms on startup.
5. **Build:** Run: mvn clean package -DskipTests to produce target/hotel_management_system-0.0.1-SNAPSHOT.jar.
6. **Run:** java -jar target/hotel_management_system-0.0.1-SNAPSHOT.jar
7. **Access:** Open http://localhost:8080 and log in with the seeded admin credentials.

7.2 Environment Variables

For production, sensitive values should be externalised as environment variables rather than stored in application.properties:

# Recommended production environment variables
DB_URL=jdbc:mysql://your-db-host:3306/hotel_db DB_USERNAME=hotel_app_user
DB_PASSWORD=<strong-random-password> SERVER_PORT=8080

7.3 Monitoring & Maintenance

Spring Boot Actuator endpoints (/actuator/health, /actuator/metrics) can be enabled for health checks. Application logs are written to stdout; redirect to a file or a log aggregation service (e.g. Papertrail, Datadog) in production.

MySQL automated backups should be scheduled nightly using mysqldump or the hosting provider's snapshot feature.

The @CreationTimestamp and @UpdateTimestamp annotations on entities provide built-in audit timestamps for data integrity verification.

8. User Guide

8.1 Getting Started

> **Default Admin Credentials** (seeded on first run): Email: admin@hotel.com | Password: admin123
> Change this password immediately after first login in production.

After logging in, the dashboard shows today's occupancy, pending housekeeping tasks, recent bookings, and low-stock inventory alerts. The navigation menu adapts based on your role: an ADMIN sees all sections; a HOUSEKEEPING user sees only their tasks.

8.2 Core Workflows

Making a Reservation

Navigate to Bookings → New Booking.
Enter guest details (name, email, phone, ID type and number). Select room category and available room.
Set check-in and check-out dates.
Choose payment method (Cash or Online).
Click Create Booking. Status is set to RESERVED.

Guest Check-in

Navigate to Bookings → find the RESERVED booking.
Click Check In. Status transitions to CHECKED_IN; room becomes OCCUPIED. The actual check-in timestamp is recorded automatically.

Guest Check-out & Billing

Navigate to Bookings → find the CHECKED_IN booking. Click Check Out. Room transitions to CLEANING.
System automatically generates a bill: room charges (nights × rate) + food charges. Navigate to Bills to view the itemised invoice and mark it as PAID.

Managing Housekeeping

Navigate to Housekeeping → New Task.
Assign a room, task type (CLEANING / INSPECTION / MAINTENANCE), and staff member. Housekeeping staff update task status through their dashboard.
When marked DONE, the completion timestamp is recorded.

Inventory Management

Navigate to Inventory to view all stock items.
Items below their low_stock_threshold are highlighted in the dashboard.

Update quantities after receiving deliveries.

9. Conclusion & Reflection

9.1 Project Summary

The Hotel Management System successfully delivers a role-aware, full-stack hotel operations platform. All primary features — guest management, room booking with conflict detection, check-in/check-out lifecycle, automatic billing, food order tracking, housekeeping management, and inventory control — are implemented and tested. The application follows established design patterns (MVC, Repository, DTO) and industry-standard security practices (BCrypt, RBAC, parameterised queries).

9.2 Lessons Learned

State machine design pays off early: Defining the booking lifecycle (RESERVED → CHECKED_IN → CHECKED_OUT) as an explicit state machine before writing code prevented many edge-case bugs. Future features should always begin with a state diagram.

BigDecimal for all monetary values: Using double for currency causes rounding errors that compound across many transactions. Always use BigDecimal with defined scale for financial calculations.

DTOs protect the API contract: Separating request/response DTOs from entity classes allowed us to change the database schema without breaking API consumers. The extra mapping code is worth the flexibility.

Spring Security configuration requires careful ordering: The security filter chain rules are evaluated in order. Placing .anyRequest().authenticated() before more specific rules accidentally blocked public endpoints. Rules must be ordered from most specific to least specific.

Database seeding simplifies onboarding: DatabaseSeeder.java that creates default staff, room categories and sample rooms on startup eliminates manual setup steps and makes the project immediately runnable for reviewers.

9.3 Future Enhancements

Automated unit and integration tests (JUnit 5 + Mockito + Spring Boot Test).

JWT-based stateless authentication to support horizontal scaling and mobile API clients. Re-enable CSRF protection for production deployment.

Booking confirmation email notifications using Spring Mail / SendGrid.

Online payment gateway integration (Stripe, PayPal) for ONLINE payment method. Interactive floor map showing real-time room status.

Revenue and occupancy reports with date-range filtering and CSV export. Docker Compose file for one-command local setup (Spring Boot + MySQL containers). Audit log table recording every status change with actor, timestamp, and previous value.